

# Sentinel Fix Pipeline - Plan, Build, Execute, Test

Neural Nexus · Smart India Hackathon 2026 · From proven findings to verified fixes

---

## The pipeline in 30 seconds

Sentinel scans and proves vulnerabilities. A converter agent turns every finding into one standard JSON work order. That JSON flows through four agent stages - Plan, Build, Execute, Test - and only verified fixes reach the user, who approves each diff before anything is applied.

## Stage 0 - Sentinel findings in

Input is the proven scan result: title, severity, file and line, proof payload, bug family, and money impact. Nothing unproven enters the pipeline, so agents never chase false alarms.

## Stage 1 - Converter agent (problems to JSON)

One agent normalizes every finding into the same JSON work-order shape: id, title, severity, score, file, line, category, proof, fix cost, and repair hours. This single format is the contract - every later stage reads JSON and writes JSON, so stages stay independent and auditable.

## Stage 2 - Plan agents (make the battle plan)

Planners read the full JSON list and produce an ordered fix plan: criticals first, grouped by file so one file is touched once, with dependencies noted (for example auth checks before route hardening). Output is a plan JSON each item can trace back to its finding id.

## Stage 3 - Build agents (find and fix everything)

Builders take one planned item at a time with full context: the file content, the proof, plus every warning, bug, error, and broken-logic signal around it. Each writes a minimal diff - parameterized queries for injection, output encoding for cross-site scripting, vault reads for secrets - never whole-file rewrites.

## Stage 4 - Execute agents (apply safely)

Executors apply approved diffs onto the practice copy only, one by one, keeping the original safe and every step reversible. Nothing touches the user's real repository without explicit approval.

## Stage 5 - Test agents (prove it stayed fixed)

Testers re-run the original proof payload plus warning, bug, error, and broken-logic checks against the patched copy. Clean item gets a green tick. Anything still failing loops back to Build with the failure attached, up to two retries, then escalates to a human with the full trail.

## End-to-end flow

Scan proves bugs. Converter writes JSON. Plan orders the work. Build writes diffs. Execute applies them on the copy. Test re-attacks each fix. User sees proven findings become green-ticked diffs, then applies or reverts. A fix counts as done only when the original exploit no longer works.

## Safety rules

Practice copy only. Minimal diffs. Human approves before anything real changes. Every stage logged against finding ids, so any fix can be traced from proof to plan to patch to passing test.

## The JSON work order (contract between stages)

```
{
  "id": "SEN-001",
  "title": "Search box lets a hacker read the whole database",
  "severity": "Critical",
  "cvss": 9.8,
  "file": "routes/login.ts",
  "line": 42,
  "category": "sql_injection",
  "proof": "email ` ' OR 1=1--` returned admin token",
  "fixCostInr": 105000,
  "fixHours": 14,
  "status": "planned | patched | verified | failed"
}
```